



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

Name of Student: Aryan Singh Kasera

PRN: 25070122052

Subject: Programming with Java

Sem: III AY: 2026-27

Assignment No: 4

TITLE: Develop a Java application demonstrating the use of wrapper classes for conversion between primitive data types and objects, and performing basic operations on wrapped values.

Problem Statement

Develop a Java application demonstrating the use of wrapper classes for conversion between primitive data types and objects, and performing basic operations on wrapped values.

Learning Objectives

To perform conversion between primitive data types and wrapper objects using Java wrapper classes.

Theory

Wrapper classes are predefined classes that encapsulate primitive data types into objects. Java provides wrapper classes such as Integer, Double, Character, and Boolean for every primitive type. They support autoboxing, unboxing, parsing, and conversion methods. Wrapper classes are widely used in Java Collections Framework and when object representation of primitive values is required.

1.Create a program to convert student marks from String format to Integer and calculate total marks.

StudentMarks.java

 Share 

```
1 class StudentMarks {
2     public static void main(String[] args) {
3         String m1 = "45";
4         String m2 = "99";
5         String m3 = "68";
6
7         Integer mark1 = Integer.valueOf(m1);
8         Integer mark2 = Integer.valueOf(m2);
9         Integer mark3 = Integer.valueOf(m3);
10
11         int total = mark1 + mark2 + mark3;
12
13         System.out.println("Mark 1: " + mark1);
14         System.out.println("Mark 2: " + mark2);
15         System.out.println("Mark 3: " + mark3);
16         System.out.println("Total Marks: " + total);
17     }
18 }
```

OUTPUT

Output

```
Mark 1: 45
Mark 2: 99
Mark 3: 68
Total Marks: 212

=== Code Execution Successful ===
```

2. Develop an Employee Payroll System that accepts employee IDs, basic salary, and bonus amounts from the user. Convert the entered values into wrapper objects and perform validation operations to ensure valid salary values before calculating the net salary

EmployeePayroll.java

 Share  Run

```
1 class EmployeePayroll {
2     public static void main(String[] args) {
3         String id = "101";
4         String salary = "50000";
5         String bonus = "10000";
6
7         Integer empId = Integer.valueOf(id);
8         Double basicSalary = Double.valueOf(salary);
9         Double bonusAmount = Double.valueOf(bonus);
10
11        Double netSalary = basicSalary + bonusAmount;
12
13        System.out.println("Employee ID: " + empId);
14        System.out.println("Basic Salary: " + basicSalary);
15        System.out.println("Bonus: " + bonusAmount);
16        System.out.println("Net Salary: " + netSalary);
17    }
18 }
```

OUTPUT

Output

```
Employee ID: 101
Basic Salary: 50000.0
Bonus: 10000.0
Net Salary: 60000.0

=== Code Execution Successful ===
```

Conclusion:

This experiment demonstrated the use of Java wrapper classes to convert primitive data types into objects (boxing/autoboxing) and objects back into primitive values (unboxing). It also showed how wrapper classes provide useful methods for parsing, type conversion, and performing operations on wrapped values. Overall, wrapper classes simplify data handling, improve compatibility with Java Collections and APIs, and make programming more flexible and efficient when object representation of primitive data types is required.